



Instituto Politécnico Nacional

Escuela Superior de Cómputo

ADMINISTRACIÓN DE SERVICIOS EN RED

Actividad:

REPORTE DE PRACTICA 5 GNS3

Alumno:

Sigala Morales Said

2022630413

Grupo: 6CM4

Fecha de realización: 29 de octubre de 2024

Fecha de entrega: 29 noviembre de 2024



1. DEFINICIÓN DEL JUEGO DEL RELOJ EN AJEDREZ. ¡Error! Marcador no definido.
2. REGLAS DE INFERENCIA DEL JUEGO..... ¡Error! Marcador no definido.
3. REGLAS DE INFERENCIA EN SENTENCIAS PROPOSICIONALES. ¡Error! Marcador no definido.



1. CONFIGURACIÓN AUTOMÁTICA DE SSH

Se utilizó el siguiente script de Python para conectarse vía Telnet a cada router y hacer el levantamiento de SSH en cada uno de ellos, para esto, se utilizó una API-REST con las librerías pexpect y Flask, las cuales corrieron en el servidor API-REST y cuando se conectaba un usuario y hacia una petición de tipo GET al servidor, se iniciaba la configuración, el comando para conectarse desde el cliente es:

```
curl -X POST http://192.168.122.100:5000/ssh -H "Content-Type:  
application/json" -d '{"username": "sshuser", "password": "sshpass"}
```



EL SCRIPT REFERIDO ES:

```
from flask import Flask, jsonify, request
import pexpect
import time
from typing import List, Dict

app = Flask(__name__)

class RouterConfig:
    def __init__(self, ip: str, username: str = "admin", password: str = "admin"):
        self.ip = ip
        self.username = username
        self.password = password

    def configure_ssh(self, new_username: str, new_password: str) -> Dict[str, str]:
        """Configura SSH en un router mediante Telnet usando pexpect."""
        try:
            print(f"[INFO] Conectando a {self.ip} mediante Telnet...")
            child = pexpect.spawn(f'telnet {self.ip}')
            child.expect('Username:')
            child.sendline(self.username)
            child.expect('Password:')
            child.sendline(self.password)
            child.expect('#')
            time.sleep(1)

            # Entrar en modo configuración
            print(f"[INFO] Configurando SSH en {self.ip}...")
            child.sendline('configure terminal')
            child.expect('#')
            time.sleep(1)

            # Lista de comandos para configurar SSH
            commands = [
                'ip domain-name local.domain',
                'ip ssh rsa keypair-name sshkey',
                'crypto key generate rsa usage-keys label sshkey modulus 2048',
                f'username {new_username} privilege 15 secret {new_password}',
                'line vty 0 4',
                'transport input ssh telnet', # Permitimos telnet también
                'login local',
                'exit',
                'ip ssh version 2',
                'ip ssh authentication-retries 3',
                'ip ssh time-out 60',
                'end',
            ]
```



```
]

for cmd in commands:
    print(f"[INFO] Ejecutando comando en {self.ip}: {cmd}")
    child.sendline(cmd)
    # Espera adicional para la generación de claves RSA
    if 'crypto key generate' in cmd:
        child.expect('The name for the keys will be:', timeout=30)
        child.sendline()
        child.expect('#', timeout=60)
    else:
        child.expect('#', timeout=5)

    # Cerrar conexión
    child.sendline('exit')
    child.close()
    print(f"[SUCCESS] SSH configurado exitosamente en {self.ip}")

    return {"status": "success", "message": f"SSH configured successfully on {self.ip}"}

except pexpect.ExceptionPexpect as e:
    error_message = f"Error configuring SSH on {self.ip}: {str(e)}"
    print(f"[ERROR] {error_message}")
    return {"status": "error", "message": error_message}
except Exception as e:
    error_message = f"Unexpected error on {self.ip}: {str(e)}"
    print(f"[ERROR] {error_message}")
    return {"status": "error", "message": error_message}

app.route('/ssh', methods=['POST'])
def configure_ssh_all():
    """Endpoint para configurar SSH en todos los routers."""
    print("[INFO] Conexión entrante a /ssh: Iniciando configuración de SSH en los routers...")
    try:
        data = request.get_json()
        new_username = data.get('username', 'sshuser')
        new_password = data.get('password', 'sshpass')

        # Lista de IPs de los routers
        router_ips = [
            '192.168.122.201', # R201
            '192.168.122.202', # R202
            '192.168.122.203', # R203
            '192.168.122.204' # R204
        ]

        results = []
        for ip in router_ips:
            router = RouterConfig(ip)
```



```
results = []
for ip in router_ips:
    router = RouterConfig(ip)
    result = router.configure_ssh(new_username, new_password)
    results.append({
        "router": ip,
        "status": result["status"],
        "message": result["message"]
    })

print("[INFO] Configuración SSH completada en todos los routers")
return jsonify({
    "status": "completed",
    "results": results
})

except Exception as e:
    error_message = str(e)
    print(f"[ERROR] {error_message}")
    return jsonify({
        "status": "error",
        "message": error_message
    }), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
root@API-REST:~#
```



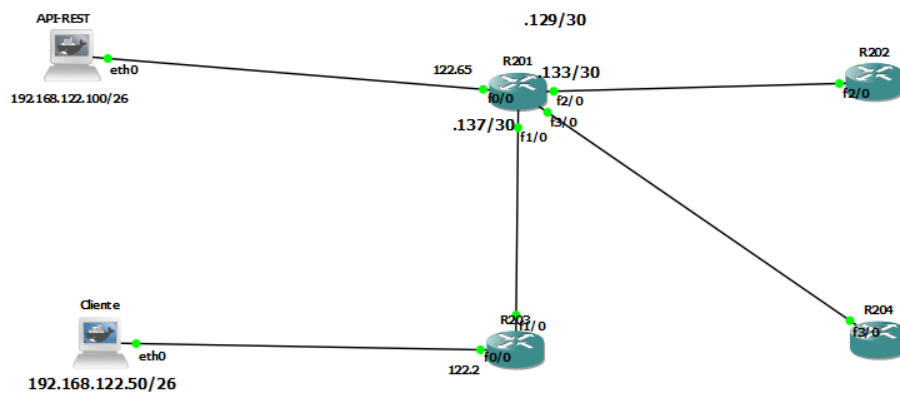
2. TOPOLOGÍA

Instituto Politécnico Nacional

Escuela Superior de Cómputo
ADMINISTRACIÓN DE SERVICIOS EN RED



IP-Administrativa
R201: 192.168.122.201
R202: 192.168.122.202
R203: 192.168.122.203
R204: 192.168.122.204



Topología Refería en la practica con sus interfaces e ips configuradas.